DEFINITION In ***token-based authentication***, a user's real credentials are presented once, and the client is then given a short-lived *token*. A token is typically a short random string that can be used to authenticate API calls until the token expires.

4.1 Authentication in web browsers

In chapter 3, you learned about HTTP Basic authentication, in which the username and password are encoded and sent in an HTTP `Authorization` header. An API on its own is not very user friendly, so you'll usually implement a user interface (UI) on top. Imagine that you are creating a UI for Natter, that will use the API under the hood but create a compelling web-based user experience on top. In a web browser, you'd use web technologies such as HTML, CSS, and JavaScript. This isn't a book about UI design, so you're not going to spend a lot of time creating a fancy UI, but an API that must serve web browser clients cannot ignore UI issues entirely. In this first section, you'll create a very simple UI to talk to the Natter API to see how the browser interacts with HTTP Basic authentication and some of the drawbacks of that approach. You'll then develop a more web-friendly alternative authentication mechanism later in the chapter. Figure 4.1 shows the rendered HTML page in a browser. It's not going to win any awards for style, but it gets the job done. For a more in-depth treatment of the nuts and bolts of building UIs in JavaScript, there are many good books available, such as Michael S. Mikowski and Josh C. Powell's excellent *Single Page Web Applications* (Manning, 2014).